

Sales Return API — LoagmaPMS

Base URL (production): `https://loagmapms-hd5u.onrender.com/api`

All endpoints require `Authorization: Bearer <token>` .

Overview

A sales return allows a customer to return items from a previously billed invoice. There is **no separate returns table** — return data is stored as columns on the same `orders` and `orders_item` rows as the original invoice.

Key rule: A sales return can only be created against an order that has a `bill_no` (i.e. is already invoiced). Only one return document can exist per order — to change quantities, use PUT to update the existing return.

Database Columns Affected

`loagma_new.orders` (header)

Column	Type	Set by return flow
<code>Sales_Return_VoucherNo</code>	<code>VARCHAR(100)</code>	Return voucher number e.g. <code>SR/25-26/001</code> . <code>NULL</code> = no return yet
<code>Sales_Return_Dt</code>	<code>DATE</code>	Date of the return
<code>Sales_Return_Reason</code>	<code>TEXT</code>	Reason text (nullable)
<code>order_state</code>	<code>VARCHAR</code>	Set to <code>DRAFT</code> / <code>POSTED</code> / <code>CANCELLED</code> by the return flow

loagma_new.orders_item (line items)

Column	Type	Set by return flow
qty_returned	INT	Quantity returned for each item — updated on every save

Read-only by return flow (never written):

- quantity — original ordered qty (set by order form)
- qty_loaded — invoice qty (set by invoice form)
- qty_delivered — delivery qty (set by delivery flow)

Voucher Number Format

Format: SR/YY-ZZ/NNN

- SR — static prefix
- YY-ZZ — financial year (April start), e.g. 25-26 for FY 2025–2026
- NNN — sequential 3-digit number within the FY, e.g. 001 , 002

Example: SR/25-26/003

The number is count-based within the current FY prefix — resets each April.

Routes

Method	Endpoint	Purpose
GET	/api/sales-returns/series	Get next available voucher number
GET	/api/sales-returns	List returns with filters
POST	/api/sales-returns	Create a new return
GET	/api/sales-returns/{id}	Get return detail (id = order_id)
PUT	/api/sales-returns/{id}	Update existing return
DELETE	/api/sales-returns/{id}	Delete return and reset all qty_returned to 0

Endpoints

GET /api/sales-returns/series

Returns the next available voucher number for the current financial year.

Response 200

```
{
  "success": true,
  "doc_no_prefix": "SR/25-26/",
  "next_number": "003",
  "full_number": "SR/25-26/003"
}
```

GET /api/sales-returns

List all returns with optional filters.

Query parameters

Param	Type	Notes
limit	int	Default 20, max 200
page	int	Default 1
search	string	Searches voucher number, customer name, order ID
customer_id	int	Filter by customer
from_date	date	Filter by return date YYYY-MM-DD
to_date	date	Filter by return date YYYY-MM-DD
status	string	DRAFT / POSTED / CANCELLED

Response 200

```

{
  "success": true,
  "data": [
    {
      "id": 268082,
      "voucher_no": "SR/25-26/003",
      "return_dt": "2026-06-05",
      "customer_id": 1234,
      "customer_name": "Ramesh Traders",
      "reason": "Defective items",
      "status": "DRAFT",
      "total_value": 450.00
    }
  ],
  "pagination": {
    "total": 12,
    "page": 1,
    "limit": 20,
    "pages": 1
  }
}

```

`total_value` = SUM(`qty_returned` × `item_price`) computed live from `orders_item`.

POST /api/sales-returns

Create a new sales return against an existing invoice.

Rules:

- The order must exist and must have a `bill_no` (must be invoiced)
- The order must NOT already have a `Sales_Return_VoucherNo` — use PUT to edit an existing return
- At least one item with `returned_quantity` > 0 required

Request

POST https://loagmapms-hd5u.onrender.com/api/sales-returns

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "source_sales_invoice_id": 268082,
  "doc_date": "2026-06-05",
  "reason": "Defective items",
  "status": "DRAFT",
  "items": [
    {
      "source_sales_invoice_item_id": 8842,
      "returned_quantity": 3
    }
  ]
}
```

Request fields

Field	Type	Required	Notes
source_sales_invoice_id	int	Yes	The order_id of the invoiced order being returned against. Also accepted as source_order_id
doc_date	date	No	Return date YYYY-MM-DD . Defaults to today
reason	string	No	Reason for the return
status	string	No	DRAFT (default) / POSTED / CANCELLED
items	array	Yes	Min 1 item

Item fields

Field	Type	Required	Notes
source_sales_invoice_item_id	int	Yes	The item_id from orders_item .Also accepted as order_item_id or item_id
returned_quantity	float	Yes	Quantity being returned. Also accepted as return_qty

What the server writes (inside a transaction):

1. Generates voucher number SR/25-26/003
2. Updates orders :
 - Sales_Return_VoucherNo = 'SR/25-26/003'
 - Sales_Return_Dt = '2026-06-05'
 - Sales_Return_Reason = 'Defective items'
 - order_state = 'DRAFT'
3. For each item, adds to qty_returned (additive):

```
UPDATE orders_item
SET qty_returned = COALESCE(qty_returned, 0) + 3
WHERE item_id = 8842 AND order_id = 268082
```

Response 200 — same as GET /api/sales-returns/{id}

GET /api/sales-returns/

Get full return detail. {id} is the order_id .

Response 200

```

{
  "success": true,
  "data": {
    "id": 268082,
    "voucher_no": "SR/25-26/003",
    "doc_no_prefix": "SR/25-26/",
    "return_dt": "2026-06-05",
    "source_order_id": 268082,
    "source_si_number": "INV/25-26/003",
    "customer_id": 1234,
    "customer_name": "Ramesh Traders",
    "reason": "Defective items",
    "status": "DRAFT",
    "items": [
      {
        "order_item_id": 8842,
        "source_sales_invoice_item_id": 8842,
        "product_id": 42,
        "product_name": "Basmati Rice",
        "product_code": "10063090",
        "unit": "Kg",
        "original_quantity": 10,
        "available_quantity": 7,
        "returned_qty": 3,
        "returned_quantity": 3,
        "unit_price": 45.00
      }
    ]
  }
}

```

Item field notes

Field	Source	Notes
original_quantity	orders_item.quantity	What was originally ordered
available_quantity	computed	$\max(0, \text{qty_delivered} - \text{qty_returned})$ — how much can still be returned
returned_qty / returned_quantity	orders_item.qty_returned	Both fields return the same value

Field	Source	Notes
unit_price	orders_item.item_price	Price per unit at invoice time

PUT /api/sales-returns/

Update an existing return. Replaces all item return quantities (PUT semantics — not additive).

Request — same shape as POST (without `source_sales_invoice_id`)

```
{
  "doc_date": "2026-06-05",
  "reason": "Updated reason",
  "status": "POSTED",
  "items": [
    {
      "source_sales_invoice_item_id": 8842,
      "returned_quantity": 5
    }
  ]
}
```

What the server writes (inside a transaction):

1. Resets ALL `qty_returned = 0` for this order's items
2. Sets each item in the payload to the exact provided value:

```
UPDATE orders_item SET qty_returned = 5 WHERE item_id = 8842 AND order_id = 268082
```

3. Updates header: `Sales_Return_Dt` , `Sales_Return_Reason` , `order_state`
4. `Sales_Return_VoucherNo` is **never changed** on update

Note: Items omitted from the PUT payload will have `qty_returned = 0` after the update — always send the complete list of items on every PUT.

Response 200 — same as `GET /api/sales-returns/{id}`

Error responses

Code	Reason
404	Order not found or has no return voucher

DELETE /api/sales-returns/

Deletes the return — resets all quantities and clears the voucher from the order.

What the server writes (inside a transaction):

1. Sets all `orders_item.qty_returned = 0` for this order
2. Updates `orders` :
 - `Sales_Return_VoucherNo = NULL`
 - `Sales_Return_Dt = NULL`
 - `Sales_Return_Reason = NULL`
 - `order_state = 'billed'` (reverts to invoiced state)

Response 200

```
{ "success": true, "message": "Sales return deleted" }
```

Item Quantity Fields Reference

DB Column	Meaning	Set by
<code>orders_item.quantity</code>	Original ordered quantity	Order create/edit
<code>orders_item.qty_loaded</code>	Invoice quantity (loaded at dispatch)	Invoice form
<code>orders_item.qty_delivered</code>	Confirmed delivered quantity	Delivery confirmation flow
<code>orders_item.qty_returned</code>	Returned quantity	Sales return flow
<code>available_quantity</code> (computed)	<code>qty_delivered - qty_returned</code>	How much can still be returned

Flutter Usage

```
// Load items from existing invoice for return form
final response = await http.get(
  Uri.parse('${ApiConfig.salesOrders}/${invoiceOrderId}'),
  headers: AuthController.getHeaders,
);
// Use items[].available_quantity to cap user input

// Create return
final response = await http.post(
  Uri.parse(ApiConfig.createSalesReturn),
  headers: AuthController.jsonHeaders,
  body: jsonEncode({
    'source_sales_invoice_id': orderId,
    'doc_date': '2026-06-05',
    'reason': 'Defective',
    'status': 'DRAFT',
    'items': [
      { 'source_sales_invoice_item_id': 8842, 'returned_quantity': 3 }
    ],
  }),
);

// Update return
await http.put(
  Uri.parse('${ApiConfig.salesReturns}/${orderId}'),
  headers: AuthController.jsonHeaders,
  body: jsonEncode({ 'status': 'POSTED', 'items': [...] }),
);
```